

Assignment: Object Detection Model Enhancement – SSD MobileNetV2

For this project, I worked to improve the performance of an object detection model within limited computational resources, using the SSD MobileNetV2 architecture as the base. I selected the Oxford-IIIT Pet Dataset because it is lightweight, easy to load using TensorFlow Datasets, and contains clean, labeled images suitable for object detection tasks. This dataset includes 37 pet breeds, but for the purpose of this project, I focused on a simplified binary classification identifying whether the object detected was a cat or a dog. This allowed me to train faster, reduce model complexity, and meet the hardware constraints of the free-tier Colab environment.

Each image in the dataset included a bounding box around the pet's head labeled as head bbox, which I used as the primary detection target. This kept the object detection task relevant while allowing me to demonstrate bounding box regression. During preprocessing, all images were resized to 224x224 to match the MobileNetV2 input dimensions and normalized to a [0, 1] pixel value range. I created a TensorFlow data pipeline to handle image resizing, normalization, batching, and prefetching for training efficiency.

I built a custom Keras model using MobileNetV2 as the feature extractor, loaded with pretrained ImageNet weights and frozen during training. On top of this, I added two heads: one for bounding box regression (predicting four coordinates per image using sigmoid activation) and one for classification (predicting either cat or dog using softmax). This dual-head approach allowed the model to learn both object localization and class prediction in a single pass. The model was trained over five epochs and saved in the standard HDF5 (.h5) format using Keras's `model.save()` method for compatibility.

Training results were very strong. The model achieved a final training accuracy of **99.8%** for classification with a training bounding box loss of **0.0230**. Validation accuracy remained high at **99.18%**, while the bounding box validation loss was around **0.2226**, showing consistent performance across both tasks. The model demonstrated that even with minimal fine-tuning and a lightweight architecture, high accuracy and efficient learning are achievable with the right preprocessing and design choices.

Throughout this project, I learned how to use transfer learning effectively with limited hardware resources. I gained hands-on experience with dual-output models, bounding box formatting, and image preprocessing using TensorFlow's `tf.data` pipeline. I also deepened my understanding of how to freeze layers, create custom heads, and train detection models end-to-end. This process gave me a much clearer picture of what's involved in real-world object detection work, and it showed me the balance between model size, speed, and accuracy.

If I had more time or access to greater computational power, I would fine-tune the deeper layers of the MobileNetV2 backbone and implement additional data augmentation techniques like horizontal flips, brightness adjustment, and rotations to further improve generalization. I'd also consider switching to a dataset with full-body bounding box annotations, or experimenting with alternate lightweight detectors like EfficientDet Lite or TinyYOLO for comparison.

This project helped me grow as both a developer and a researcher. I now feel more confident working with object detection models, and I know how to get high performance even when resources are tight. This report, along with my model and notebook, demonstrates a full pipeline from dataset selection and preprocessing to training, evaluation, and reflection.